

Porting the ICON 2-Moment Microphysics to GT4Py

Benjamin Buchenau, Jana Celidze-Eder, Pascal Stöckli, Aljoscha Wrona

September 15, 2026

1 Why move away from a Fortran framework?

ICON, like many numerical weather prediction models, relies on a large Fortran code base. Fortran is well suited for numerical computation and can achieve high performance, but modern HPC systems increasingly combine different CPU and GPU architectures. Efficiently using these systems often requires architecture-specific programming models, compiler directives, or specialised implementations. Maintaining and optimising several versions of the same scientific code therefore increases the development effort. This contributes to the software productivity gap discussed by Ubbiali et al. (2025): while hardware becomes increasingly heterogeneous, scientific models also continue to grow in size and complexity.

In our project, we investigate this problem for the Seifert and Beheng (2006) two-moment microphysics scheme, which forms part of the ICON NWP physics. The scheme describes the evolution and interaction of several hydrometeor categories through processes such as cloud activation, nucleation, freezing, deposition, collisions, melting, evaporation, and sedimentation. Consequently, the scientific formulation is already complex without also considering the details required to optimise the code for a particular architecture.

We therefore want to separate the description of the physical processes from hardware-specific implementation details. The aim is not to replace Fortran because it performs poorly, but to investigate whether a higher-level programming model can make the physics easier to maintain while still allowing efficient execution on different HPC architectures.

To make this investigation manageable, we first isolated the microphysics from the full ICON model and created a column-model setup. This allows us to run the scheme independently and compare the port directly against the original Fortran implementation. We then port increasingly complex, physically consistent subsets of the scheme instead of attempting the complete microphysics at once.

2 What we want from GT4Py

With GT4Py, we want to retain the performance required for numerical weather prediction while expressing the scientific computation at a higher level - with the hope of making the code more readable and maintainable by domain scientists. GT4Py provides a Python-based domain-specific programming model for weather and climate applications. Instead of explicitly implementing low-level loops and hardware-specific parallelisation, it allows us to describe the numerical operations in Python and rely on the GT4Py toolchain to generate code for different backends (Ubbiali et al., 2025). Previous work applying GT4Py to ECMWF microphysics schemes shows that this approach can provide competitive performance across heterogeneous HPC systems (Ubbiali et al., 2025). We use the ICON two-moment microphysics as a further test case for this programming model.

Finally, we want to understand where the GT4Py abstraction works well and where it reaches its limits. We therefore evaluate GT4Py not simply as a replacement for Fortran, but as a programming model for realistic NWP physics.

3 Setup: Isolating microphysics from ICON

3.1 Base setup: The column model

In order to be able to port the Seifert and Beheng (2006) two-moment microphysics scheme (SB06 hereafter) efficiently, we need to be able to quickly run it in isolation in order to verify the ported code against the reference. In addition to removing the overhead of running the full model, it will be practical to have a stand-alone driver so that we do not need to worry about interfacing our GT4Py implementation with the rest of ICON.

We therefore set up a column-model version of the microphysics. This means removing all looping over horizontal grid points, hoisting the microphysics call out of the ICON physics driver loop, and providing the necessary input/output scaffolding in a top-level driver. That top-level driver also handles the initialisation of the scheme to set its configuration options. Because initialisation is hardcoded, the model is only set up to run a single time step (anything else would be physically nonsensical either way).

The treatment of cloud droplet condensation in the column model requires further explanation: as is common for bulk microphysics schemes, SB06 does not treat this process, leaving it to a separate, stand-alone saturation adjustment routine. Saturation adjustment is based on the assumption that cloud droplets equilibrate very quickly with their environment. Therefore, the saturation adjustment routine evaporates or grows cloud droplets such that the air is brought to saturation with respect to liquid water. In the full ICON model, saturation adjustment is called once before the fast physics call (to respond to changes in supersaturation due to the dynamics) and once after the microphysics call (to account for changes in supersaturation due to the microphysics). Arguably - despite its omission in SB06 - cloud droplet condensation is part of the microphysics. Consequently, we mimic ICON in the column model by

wrapping the call to SB06 in two saturation adjustment calls.

3.2 Limitations of the column-model setup

Of course, a single-timestep setup cannot reveal whether small numerical discrepancies between the ported and reference code would accumulate or drift over many timesteps in a real forecast run. The column model setup also means that we cannot test the port in the context of the full-grid model.

3.3 Hierarchy of complexity

SB06 is a large parametrisation comprising many sub-processes. For this reason, we decided to start with porting smaller, physically consistent subsets of the scheme. This allowed us to gain intuition for the overall structure required for the port, and for how to best manage the agent in the process.

The least complex subset - which we started with - comprises just the saturation adjustment routine. The next step is a larger subset that we refer to as ‘minimal microphysics’. Figure 1 shows the structure of the full SB06 scheme, with the components included in our minimal version highlighted in blue. This stripped-back version aims to remain physically consistent by removing all routines dealing with precipitation-sized particles. The motivation for doing so is that much of the precipitation microphysics, at first glance, appears difficult to port: sedimentation is, by default, performed using an implicit vertical solve, for which GT4Py provides only very limited support. Many of the stripped-out routines also make extensive use of gamma-distribution lookup tables, and hail wet growth requires the parsing of a netcdf file. All of this adds complexity that, in the first instance, might obscure the general performance of the agent and prevent us from focusing on the core challenges.

4 Agentic workflow

4.1 Implementation workflow

The general workflow and what was used to achieve the final version of the port is shown in Figure 2. Note that not all team members used a separate agent for prompt improvement: we applied this step for the saturation adjustment port, but not for the minimal microphysics port. We discuss the potential impacts of prompting agents in section 4.3.

For each port, we gave the agent the GT4Py repository and the Fortran source as context. The [ICON4Py](#) repository (the parts of ICON that have already been ported to GT4Py) was withheld from the agent for the saturation adjustment port, because it would already provide a solution as part of ICON4Py’s existing one-moment scheme. For the minimal microphysics and the attempted full port, we explicitly added it to the context and instructed the agent to copy the handling of constants and datatypes. The verification agent was started in a fresh context, as an agent inheriting the coding agent’s conversation also receives its misconceptions.

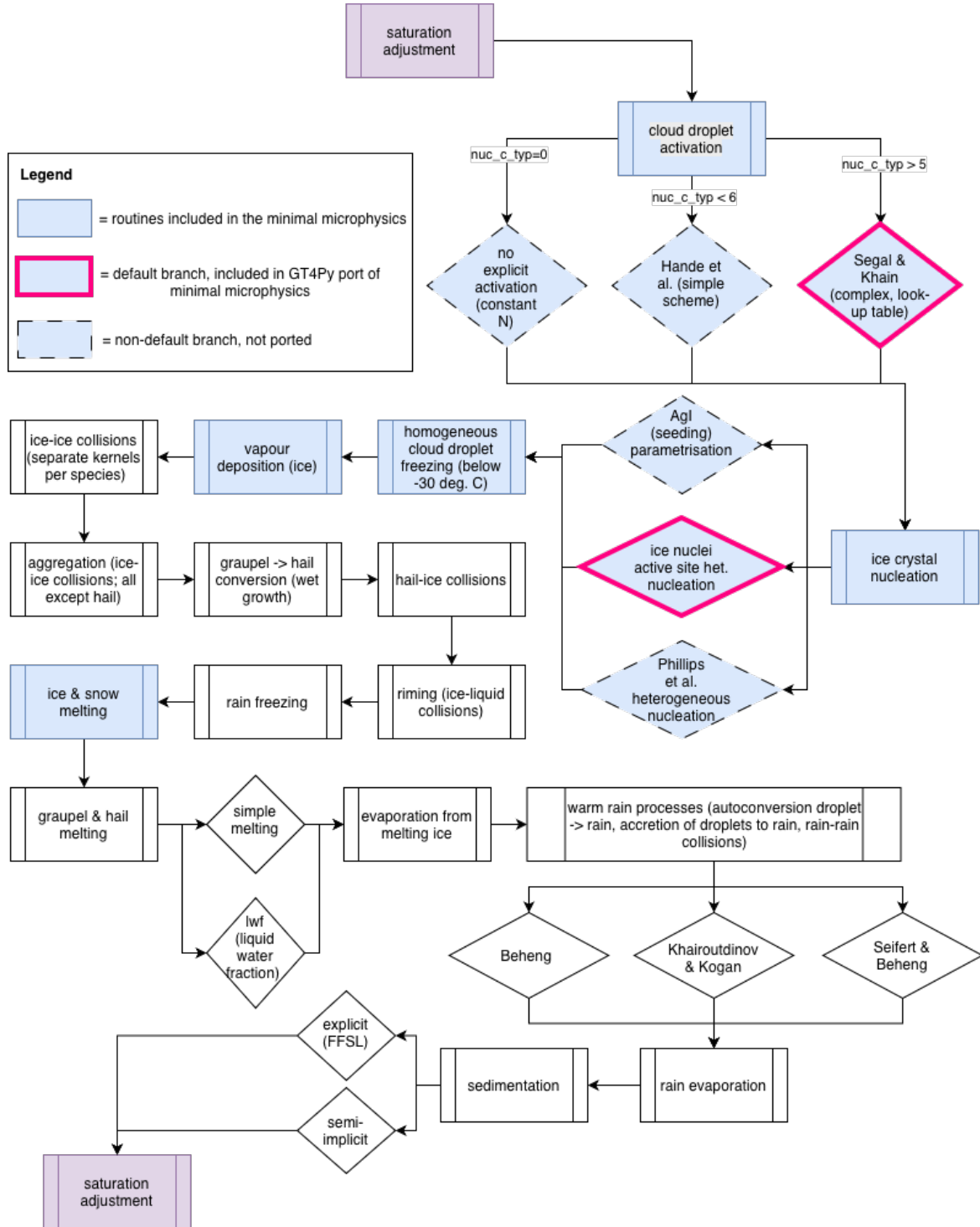


Figure 1: Flow chart of the Seifert and Beheng (2006) two-moment microphysics scheme as used in this work. The colour shading indicates which routines we include in the simplified (‘minimal’) microphysics scheme used in our porting hierarchy. Saturation adjustment is a stand-alone routine, not technically part of the microphysics, and therefore shaded purple.

Rather than a single instruction, the coding agent was given a staged protocol with explicit stopping points at which a human had to approve before work continued. Steps 1–5 expand the *porting plan* box of Figure 2, step 6 the *implementation* box. The following steps were written using the coding agent Claude Opus 5 from its history:

1. **Familiarisation with the DSL.** Read the GT4Py `field_operator` and `program` documentation and examples until the available constructs and their restrictions were established. *[approval]*
2. **Analysis of the Fortran source.** Identify the inputs, outputs, units, physical meaning, convergence logic and edge cases of the target routine. *[approval]*
3. **Design plan.** Draft the file structure, the function signatures including units and data types, and the intended treatment of those parts with no direct GT4Py equivalent (iteration, thresholds, lookup tables). Genuine uncertainties were to be flagged explicitly rather than resolved silently.
4. **Plan review.** The plan was reviewed and revised before any code was written. *[approval]*
5. **Implementation.** Write the port, documenting assumptions and interfaces such that the result can be tested without further reference to the agent’s conversation.

The ordering is deliberate. Because the restrictions of the DSL were established before the plan was drafted, constructs without a GT4Py equivalent, most importantly the `WHILE` loop of the Newton iteration, were identified at the planning stage rather than encountered during implementation.

4.2 Impact of model choice

Throughout most of the work and for all the ported code described in this document, we used Claude Code (Anthropic’s CLI-based coding agent) with Claude Sonnet 5, and Claude Opus 4.8 and 5 as the backend models. During the beginning phase of the project, we experimented with smaller open-weight models running locally on a single Grace-Hopper chip directly on Alps. We achieved a relatively stable agentic coding environment by running Qwen3.6-27B (128k context window, compared to Claude Sonnet 5 with a 1M-token window) using the Python package `vllm` and the VSCode extension Cline for agentic coding. However, we would recommend testing a command line agentic coding environment for future projects, as this might be simpler and more efficient than working with a VSCode extension. We compared both approaches for general planning tasks and the isolated port of the saturation adjustment. Although both agents showed good understanding of the Fortran code and produced reasonable output throughout the planning phase, the performance of the Qwen3.6 agent decreased over longer conversations, as the agent forgot parts of the earlier conversation due to the smaller context window. In a direct comparison, both models produced a working version of the saturation adjustment, but the Claude agent produced a cleaner code structure, an assessment both agents agreed on after

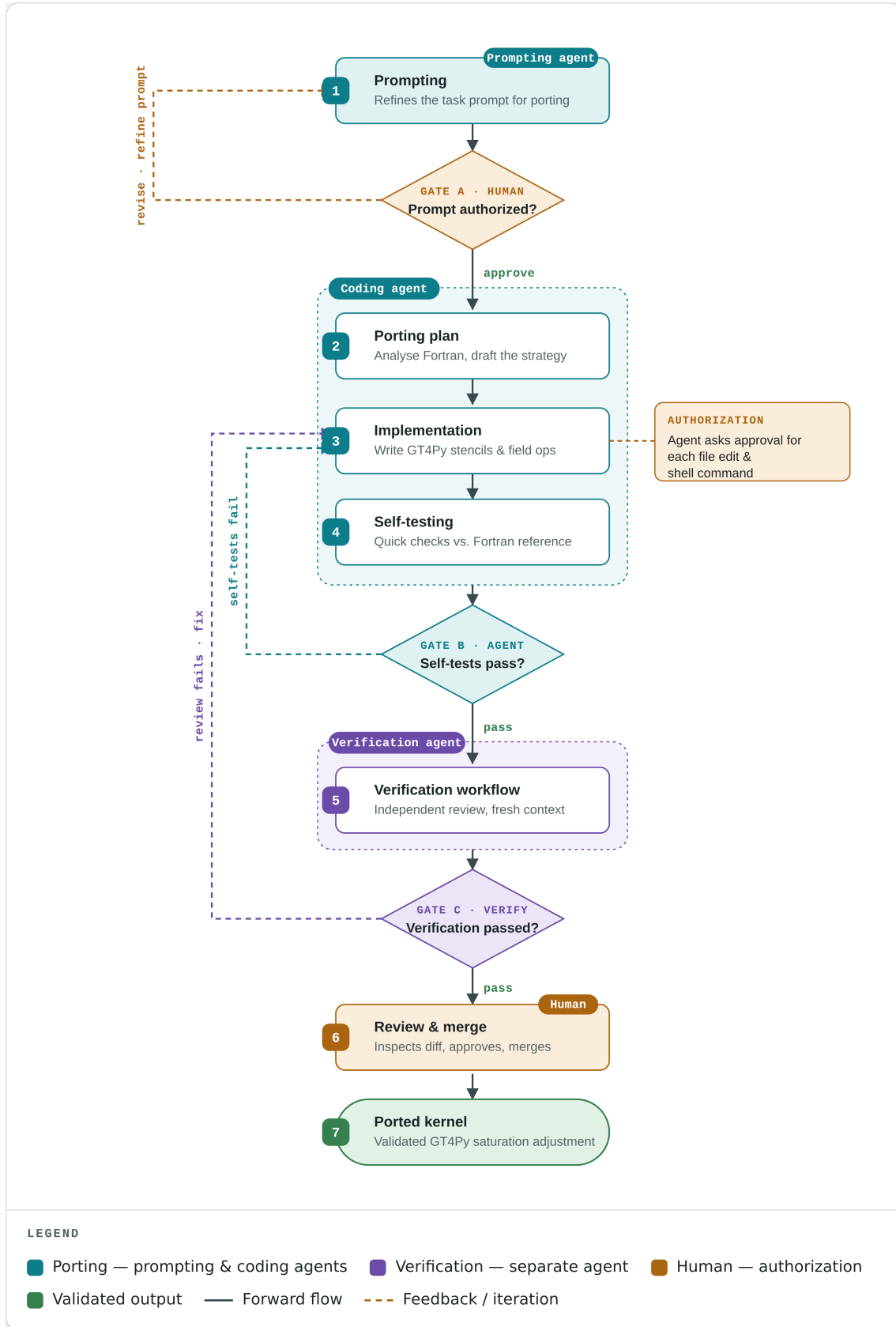


Figure 2: Implementation workflow of agentic porting project, reaching from prompting to planning, coding, stability testing, verification and final review. Note that for certain steps different agents are being used. For further details on the verification workflow the reader is referred to Section 5. This flowchart was generated using Claude Opus 4.8.

comparing the two versions. Additionally, the Qwen3.6 agent tended to solve problems by trying creative workarounds instead of conducting research about the intended usage of GT4Py. These factors led us to decide to use Claude agents only for the more complex work. Still, we think that the Qwen model showed impressive performance for its comparatively small size, and that a larger open-weight model running on several chips could be a reasonable choice for cases where sending data to external companies is not an option.

4.3 Limitations and reproducibility

As described in the previous section (4.2), the choice of agent strongly impacts code quality and workflow. However, the large language models (LLMs) are updated frequently and evolve rapidly, such that our conclusions and struggles likely become outdated after only a short period of time. This fast-paced development also limits reproducibility, and raises the question of whether strict reproducibility can even be a realistic goal in this fast-moving field. Separately, we also observed and hypothesize that the ‘Thinking’ mode of the models may introduce unexpected errors and reduce overall code quality due to more creative solutions, as also shown in the previous section. Within this frame, this issue was not further explored, only hypothesized.

Another suspected but not particularly explored limitation of this project is the impact of the prompt. We did not pursue this question partly because we do not have the required expertise and do not have insight into how to best use LLMs to get the best possible results. This is not necessarily a disadvantage unique to our study, however: other researchers attempting to reproduce our results are unlikely to be any more proficient in prompt engineering than we are. Closing this gap would require collaboration between trained prompt engineers or AI specialists, scientific researchers, and software engineers working in the field of weather and climate modeling. As one attempt to partially address this gap, an alternative approach to prompting was taken in parts of this project: for the Claude porting process, a prompt was drafted and given to another Claude agent to formulate a more suitable prompt. However, the impact of this step was not measured, though it would be an interesting study for future endeavours. Indeed, current research on this topic offers no definitive answer on how prompt design affects code quality. Della Porta et al. (2025) analysed 7583 ChatGPT-generated code files across quality metrics and found no statistically significant differences in maintainability, security, or reliability metrics across different prompt patterns. This suggests that for this set of quality dimensions, prompt structure may matter less than we assumed. In contrast, a more recent study by Rudyk et al. (2026) reveals that the effectiveness of prompt engineering techniques does not evolve uniformly across model generations, but instead ‘ages’ in a model-family-specific way. For example, newer GPT models show diminishing returns from structured prompting, while Qwen models continue to benefit from the same techniques. In summary, these mixed results suggest there is currently no stable, general understanding of how prompt design affects code quality across models, which further complicates reproducibility in this field.

In addition to this prompting-related uncertainty, we also suspect a further source of variability: session-to-session differences that are independent of prompting and

instead reflect an intrinsic non-linearity of the model itself. Ouyang et al. (2025) provide direct empirical support for this hypothesis by showing that ChatGPT can return structurally and semantically different code for the same prompt across repeated requests. Additionally, they discover that setting the temperature to its most deterministic value does not fully eliminate this variability. During our study with Claude, we had no insight into or control over these settings, so this source of variability was present throughout our project, layered on top of the prompting-related variability discussed above.

Beyond these model- and prompt-related uncertainties, a further limitation concerns the scope of our evaluation: the performance assessment between the two agents was purely qualitative. In future, more specific quality metrics could be introduced to determine the strengths and weaknesses of each agent’s output.

5 Verification

While the porting agents wrote simple regression tests as they went along, we did not want to rely on these tests alone for verification. Instead, the idea is to let a second agent write tests for the port after it has been completed, and if needed make adjustments. We decided to stick to regression tests (i.e., checking that the output of each ported process is the same as in the Fortran source) for simplicity.

The testing framework should fulfil the following criteria:

- Each process routine should be covered. This includes conditionals: often, routines exit early depending on environmental conditions (e.g., super- or subsaturation). In those cases, all branches of the conditional should be tested.
- The Fortran reference should be run on-line (rather than relying on precomputed output) so we can be sure that we are comparing like for like.
- Relative errors (i.e., $(a - b)/b$ where a is the port and b is the reference solution) should generally be within numerical roundoff error (we aim for $1\text{e-}14$). Fields that may be zero or very close to it can fall back on a (similarly small) absolute error.

To fulfil the coverage requirement, the agent needs to be able to reliably construct columns with particular conditions. We found that if left to its own devices, it just makes guesses for which combination of temperature, pressure, and humidity results in an environment with a certain supersaturation, often leading to erroneous results. To counter this, we provide the agent with a set of thermodynamic functions (saturation vapour pressure etc.) and instruct it to use these functions to construct a set of scenario columns that together exercise all code branches. Spot checks on some of the values in those columns suggest this approach produces reliable results.

For the saturation-adjustment-only port, such a testing framework is relatively simple to devise: there is only one routine to check, so our coverage requirement collapses to simply constructing columns that cover all of its branches. For the minimal microphysics, however, the task becomes slightly more complex because many different

process routines are called. To satisfy our requirement of running on-line, we ask the agent to amend the Fortran code such that we get output after each process call. This way, processes can be regression-tested in isolation without needing to write harnesses for each Fortran subroutine.

5.1 Verification outcomes: is full reproducibility desirable?

In the minimal microphysics, the verification agent uncovered some apparent issues using the framework laid out above: some output variables did not agree to the required tolerance. The agent traced this back to two issues:

- The GT4Py saturation adjustment unconditionally applies 10 Newton iterations, whereas the Fortran can stop once reaching a specified convergence criterion using a `WHILE` statement (which GT4Py does not support).
- Some constants are written out as plain floats in the Fortran reference, which makes them default to 32-bit precision, while copying the same numbers into the GT4Py port had them evaluated in 64-bit. Because they are written out to six significant digits, this alone appears to result in a test failure.

In our view, neither of these are genuine issues. The first one may be a performance issue (because the GT4Py port over-solves), but it does not affect the accuracy of the result. The ICON4Py port of saturation adjustment works around this inconsistency by defining a masking stencil that masks out converged cells after each Newton iteration. One of our agents independently arrived at a similar solution, but found this resulted in dramatically longer compile times.

The second issue has nothing to do with either physics or numerics, but just with the way numbers are stored. This raises the question whether an exact port actually makes sense, or whether it would be better to explicitly ask the agent to take differences between the languages into account.

So in summary, the agents' port appears to be sound in principle. But whether the port actually makes use of GT4Py's benefits is another question. A simple count of non-blank, non-comment lines in both implementations of the minimal microphysics seems to suggest that at least the GT4Py port is significantly more concise: The Fortran line count (removing the four unported routines) is 4366, whereas the GT4Py version numbers 2444 lines - i.e., about a 1.8x reduction.

6 Challenges of a full 2-moment microphysics port

After the successful port of the minimal-microphysics version, we attempted a port of the full 2-moment microphysics scheme. Contrary to previous work, the problems we encountered were less related to the performance of the agents, but more to the structural challenge to translate from the procedural programming paradigm in the Fortran code to the declarative GT4Py paradigm and to satisfy GT4Py's syntax restrictions. The main difficulties we faced were:

1. **Data-dependent branching:** An example to illustrate this, is the subroutine `ice_riming`, which alone contains 15 distinct conditional branches nested up to five levels deep. It is driven by combinations of process-rate thresholds, temperature regimes, particle-size criteria, and a configuration flag. A literal translation would require roughly as many masked or conditional constructs in GT4Py; accounting for all combinations, a single grid point can take on the order of 100 distinct execution paths through this one subroutine. Translating the `if/else` conditions in the Fortran code to `where()` operations in GT4Py works well in simple cases, but increases compile time and potentially influences performance when too many branches are computed over the whole field, though only needed for a few grid cells. Compiled in isolation, `ice_riming` itself is unproblematic; the compile time only becomes excessive if it is bundled with several other routines with the same sequential-branching style, suggesting that chains of dependent conditions compound compile cost, rather than branching within a single `field_operator` alone.
2. **Lookup tables:** SB06 fetches values from precomputed lookup-tables, a process not natively supported within pure GT4Py field-operators and scan-operators. In some cases, the lookup-routines could be implemented in pure Python rather than compiled GT4Py code, although this might negatively influence performance and cannot be called within larger compiled programs.
3. **Implicit vertical solvers:** The sedimentation process requires an iterative, level-by-level computation over the vertical column. GT4Py generally supports such solvers by using a `scan_operator`. However, in SB06, most microphysics routines are called within that vertical loop. The only way to preserve the data flow from the Fortran routine would be to wrap most of the microphysics and the sedimentation in one large scan-operator. This was not achievable for two reasons: lookup-table routines that cannot be called within a scan-operator and because compiling the remaining bundled microphysics did not finish within several hours. The latter appears to be tied to the data flow from the original SB06 scheme rather than to program size in general, as comparably large programs with less complicated data flow from ICON4Py’s one-moment-scheme compile well.

Because of these problems, we could not achieve a port of the full 2-moment microphysics with a compiled version of the sedimentation-solver with the current GT4Py-version. The only way we found to achieve a straight translation of the Fortran code was by implementing the lookup routines and the vertical iteration of the sedimentation routine in pure python and only calling compiled GT4Py programs below that level, which we expect to lower performance compared to a compiled scan-operator.

7 Declaration of Originality

Where indicated and reasonable, paragraphs and figures were generated using AI. Otherwise, words and figures are our own. AI was further used to assist with literature research and information extraction from said publications. In certain instances,

self-drafted paragraphs were formulated with help of AI, while ideas remain original and each output was reviewed critically and carefully. While core report structure was developed by us, AI was used for minor adjustments. We, as a team, take full responsibility over the content of this report.

References

- Della Porta, A., Lambiase, S., & Palomba, F. (2025). Do prompt patterns affect code quality? a first empirical assessment of chatgpt-generated code. *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, 181–192. <https://doi.org/10.1145/3756681.3756938>
- Ouyang, S., Zhang, J. M., Harman, M., & Wang, M. (2025). An empirical study of the non-determinism of chatgpt in code generation. *ACM Trans. Softw. Eng. Methodol.*, 34(2). <https://doi.org/10.1145/3697010>
- Rudyk, A., Oertel, J., & Hebig, R. (2026). Aging of prompt engineering techniques across LLM versions. *Proceedings of the 42nd IEEE International Conference on Software Maintenance and Evolution (ICSME)*. <https://arxiv.org/abs/2608.24641>
- Seifert, A., & Beheng, K. D. (2006). A two-moment cloud microphysics parameterization for mixed-phase clouds. Part 1: Model description. *Meteorology and Atmospheric Physics*, 92(1), 45–66. <https://doi.org/10.1007/s00703-005-0112-4>
- Ubbiali, S., Kühnlein, C., Schär, C., Schlemmer, L., Schulthess, T. C., Staneker, M., & Wernli, H. (2025). Exploring a high-level programming model for the nwp domain using ecmwf microphysics schemes. *Geoscientific Model Development*, 18, 529–546. <https://doi.org/10.5194/gmd-18-529-2025>